

EECS 111 System Software, Spring 2026

Project 3: Thread Scheduling

Due Date: May 30, 11:59 PM

Instructions

In this project, you will implement user-level scheduling decisions for periodic worker threads. The starter code already creates the worker threads, initializes their thread control blocks, manages each worker's ready/block/run loop, and simulates task execution. Your work is to complete the scheduler functions in `main.cpp` so that the main thread can choose which worker should run next.

You should read the starter code before writing your solution. Worker threads do not choose when they run. Each worker adds itself to the ready queue, blocks on its own resume condition variable, runs only after the scheduler signals it. After a worker runs, then either completes and blocks until its next period, or returns to the ready queue if preempted.

Starter Code Files

After unzipping the starter-code directory, you should see the following files:

1. `main.cpp`: program entry point, global scheduler state, task setup, and the scheduler functions you must complete.
2. `p3_threads.h`: declaration for the worker thread function.
3. `p3_threads.cpp`: worker thread implementation. This file is organized into helper functions for entering the ready queue, waiting for the scheduler, running work chunks, and handling preemption.
4. `utils.h`: utility declarations.
5. `utils.cpp`: timestamp and timed-wait helper functions.
6. `types_p3.h`: `ThreadCtrlBlk` class declaration.
7. `types_p3.cpp`: type implementation file kept for project structure.
8. `Makefile`: compilation script. The executable must be named `p3_exec`.

You only need to complete the blank scheduling functions in `main.cpp`. You may add helper functions or helper variables in `main.cpp` if needed. Do not modify the other source files and do not change the existing print statements in the starter code. The worker code in `p3_threads.cpp` is organized into helper functions so you can read the scheduling handshake without rewriting it. Your scheduler should add the required scheduler output lines described below.

Command-Line Argument

Your executable takes exactly one argument: 0, 1, or 2.

0 means FIFO scheduling.

1 means Earliest Deadline First (EDF) scheduling.

2 means Rate-Monotonic (RM) scheduling.

If the argument is missing, non-numeric, or outside the allowed range, print exactly:

[ERROR] Expecting 1 argument, but got <number>

[USAGE] p3_exec <0, 1, or 2>

Problem Description

The program models periodic real-time tasks. Each task is represented by one pthread and one ThreadCtrlBlk. The task parameters are already initialized in the starter code.

The main thread repeatedly calls the selected scheduler. The scheduler examines shared state, chooses a ready worker when appropriate, and signals that worker's resume condition variable. A worker runs in short chunks so that EDF and RM can preempt a lower-priority running worker.

Required Scheduler Functions

Your program must define and use the following functions in main.cpp:

fifo_schedule()

edf_schedule()

rm_schedule()

FIFO scheduling should select the first thread id in ready_queue when no other thread is running.

EDF scheduling should select the ready thread with the earliest absolute deadline. If a thread is already running and a ready thread has an earlier deadline, EDF should preempt the running thread and then schedule the ready thread with the earliest deadline.

RM scheduling uses fixed priority: a shorter period means higher priority. If a thread is already running and a ready thread has a shorter period, RM should preempt the running thread and

then schedule the highest-priority ready thread.

Threading and Synchronization Requirements

Your implementation must satisfy these requirements:

1. Use one pthread per worker task; do not create extra worker threads.
2. Use ready_queue to track workers that are ready to run.
3. Use running_thread to determine whether a worker is currently running. A value of -1 means no worker is running.
4. Protect shared scheduler state with mutex before reading or changing it.
5. Use pthread_cond_signal to resume a selected worker through resume[id].
6. Use the existing preempt flag and preempt_task condition variable for preemption.
7. Avoid changing the worker-thread logic in p3_threads.cpp.
8. The program must terminate after the main loop finishes and all worker threads are joined.

Preemption Guide

When EDF or RM decides that the running thread should be preempted, use the pattern below while holding mutex:

```
preempt = 1;
pthread_cond_wait(&preempt_task, &mutex);
```

The running worker checks preempt between work chunks. When it sees the flag, it prints a preemption line, sets running_thread back to -1, puts itself back into ready_queue, and signals preempt_task so the scheduler can continue.

Runtime Output

Keep all existing print statements unchanged. Add the scheduler lines exactly in the formats below when your scheduler selects or preempts a thread.

Required scheduler output formats:

```
[<Time> ms][Scheduler] FIFO selected Thread <id> (deadline <number> ms, period <number> ms)
```

```
[<Time> ms][Scheduler] EDF selected Thread <id> (deadline <number> ms, period <number> ms)
```

```
[<Time> ms][Scheduler] EDF preempts Thread <running_id> for Thread <ready_id>
```

[<Time> ms][Scheduler] RM selected Thread <id> (deadline <number> ms, period <number> ms)

[<Time> ms][Scheduler] RM preempts Thread <running_id> for Thread <ready_id>

Existing worker output lines include these events:

[<Time> ms][Thread <id>] Ready to be scheduled

[<Time> ms][Thread <id>] Starting task (iteration <number>) taking <number> ms

[<Time> ms][Thread <id>] Thread preempted

[<Time> ms][Thread <id>] Restarting task (iteration <number>) with <number> ms remaining

[<Time> ms][Thread <id>] Completed task (iteration <number>) taking <number> ms

[<Time> ms][Thread <id>] Task (iteration <number>) missed its deadline!! (Deadline: <number> ms)

The exact timestamps and interleaving may vary. The prefixes, thread ids, policy names, and required wording must remain consistent.

Conceptual Questions

Submit brief answers to the following questions in a file named questions.txt:

1. Compare FIFO, EDF, and RM scheduling for this project. Which information does each scheduler use to choose the next thread, and how can the same ready queue produce different choices under the three policies?
2. Identify the shared variables used by the scheduler and worker threads. Explain why the mutex is needed, and explain how pthread_cond_wait and pthread_cond_signal are used to block and resume worker threads.
3. Describe how your EDF and RM implementations decide when to preempt the running thread. What changes in the program state when preemption happens, and how do the required output lines help show that preemption and completion happened correctly?

Submission

Compress your source files and questions.txt into a single .zip file and upload it to Gradescope. Since submissions may be processed by a program, follow these rules carefully:

1. Keep the required output format exactly as specified.
2. Use only the C++98 standard. Do not use C++11, C++14, or newer language features.
3. Make sure your code compiles and runs with make.
4. Your executable filename must be p3_exec.

5. Your submission must contain all source code files, the Makefile, and questions.txt in the root of the uploaded zip file.
6. If creating a zip file, run the command from inside the directory containing your source code:

```
zip -j p3.zip main.cpp p3_threads.h p3_threads.cpp types_p3.h types_p3.cpp utils.h utils.cpp  
Makefile questions.txt
```
7. Your code will be checked for similarity against other submissions. You may discuss ideas and use course resources, but you must write your own code.

Grading

1. (5%) Following the submission format.
2. (10%) Compiling with the provided Makefile without errors.
3. (10%) Command-line validation and required output format.
4. (10%) Correct FIFO scheduling behavior.
5. (15%) Correct EDF scheduling behavior, including earliest-deadline selection and preemption.
6. (15%) Correct RM fixed-priority scheduling behavior, including selection and preemption; deadline misses are acceptable if the policy is implemented correctly.
7. (5%) Correct synchronization of shared scheduler state with pthread mutexes and condition variables.
8. (30%) Answers to conceptual questions.

Late Policy:

- 10% deduction if late by less than 24 hours
- 20% deduction if late by 24 - 48 hours

Note

Run your program multiple times and test all required arguments. FIFO and RM may miss deadlines on the provided workload; EDF is expected to be deadline-aware and should avoid those misses when implemented correctly. If you have questions, ask them in the discussion session or in Ed discussion.